

index.tsx

frontend/pages | TypeScript / React

```
/**  
 * index.tsx — BioSync v3 Main Dashboard  
 */  
  
/* All three signal channels converge here. The fusion engine drives UI state,  
 * and AI interventions are triggered autonomously on escalation events.  
 */  
  
/* Architecture:  
 * useRPPG → visual channel (rPPG variance score)  
 * useVocalJitter → audio channel (jitter score)  
 * hinglishParser → semantic channel (intent score, event-driven)  
 * useFusionEngine → weighted fusion → CognitiveAssessment  
 * useStateMemory → agentic failure memory (localStorage commits)  
 * getBioSyncIntervention → Groq Llama 3.1 or offline fallback  
 */  
  
'use client';  
  
import { useEffect, useRef, useState, useCallback } from 'react';  
import { motion, AnimatePresence } from 'framer-motion';  
  
import VpgGraph from '../components/VpgGraph';  
import { useFusionEngine } from '../hooks/useFusionEngine';  
import { useStateMemory } from '../hooks/useStateMemory';  
import { getBioSyncIntervention } from '../ai/groqClient';  
import { parseHinglishIntent } from '../utils/hinglishParser';  
  
// ■■■ Theme ████████████████████████████████████████████████████████████████████████████████  
  
const THEME = {  
  CALM: {  
    border: 'border-cyan-500',  
    shadow: 'shadow-cyan-500/30',  
    text: 'text-cyan-400',  
    bg: 'bg-slate-950',  
    label: 'CALM_STATE',  
    pulse: false,  
  },  
  LOAD: {  
    border: 'border-yellow-500',  
    shadow: 'shadow-yellow-500/30',  
    text: 'text-yellow-400',  
    bg: 'bg-slate-950',  
    label: 'LOAD_STATE',  
    pulse: false,  
  },  
  CRITICAL: {  
    border: 'border-red-500',  
    shadow: 'shadow-red-500/40',  
    text: 'text-red-400',  
    bg: 'bg-[#0d0203]',  
    label: 'CRITICAL_STATE',  
    pulse: true,  
  },  
} as const;  
  
// ■■■ Component ████████████████████████████████████████████████████████████████████████████████  
  
export default function BioSyncDashboard() {  
  
  // ■■ Signal hooks ████████████████████████████████████████████████████████████████████████████████  
  // semanticScore is event-driven (from speech recognition), defaulting to 0  
  const [semanticScore, setSemanticScore] = useState(0);  
  
  const {  
    assessment, // {score, state, transitioned, prevState}  
    signalConfidence,
```


[illegible]